

Symmetric functions

Math 737 - Lab 6

In this lab, we will study the algebra of symmetric functions Sym and its various bases. Evaluate the block below to define the commands for Sym and the bases we discussed in class: m, e, h, s, p .

```
In [10]: from IPython.display import Math, display
#----- jupyter doesnt display sage objects properly unless I make a
def dplay(x):
    return display(Math(latex(x)))
#-----
```

```
In [13]: Sym = SymmetricFunctions(QQ)
```

```
m = Sym.monomial()
e = Sym.elementary()
h = Sym.homogeneous()
s = Sym.schur()
p = Sym.powersum()
```

```
SymBases = {
    "monomial": m,
    "elementary": e,
    "homogeneous": h,
    "schur": s,
    "powersum": p
}
```

Exercise 1:

Express the monomial symmetric function $m(\lambda)$ for $\lambda=322$ in the basis of Schur functions using the command `s(m[3,2,2])`.

Do the same for some example elementary, homogeneous, and power sum symmetric functions (at least two of each kind).

Which bases do you think are Schur-positive?
(This means the coefficients you get when
expressing them in the Schur basis are always
positive.)

```
In [26]: partitions = [[3,2,2], [3,2,1,1],[4,3,1,1]]

for name, B in SymBases.items():

    print(f"{name.capitalize()}:")
    for lamda in partitions:
        row = []
        expr = Math(
            latex(f"{lamda}: ") +
            latex(s(B[lamda]))
        )
        row.append(expr)
        display(*row)
    print("-----")
```

Monomial:

$$\text{\texttt{[3, { }2, { }2]:{ }}} 3 s_{\{1,1,1,1,1,1\}} - 3 s_{\{2,1,1,1,1\}} + 2 s_{\{2,2,1,1,1\}} - s_{\{2,2,2,1\}} + s_{\{3,1,1,1,1\}} - s_{\{3,2,1,1\}} + s_{\{3,2,2\}}$$

$$\text{\texttt{[3, { }2, { }1, { }1]:{ }}} -12 s_{\{1,1,1,1,1,1,1\}} + 6 s_{\{2,1,1,1,1,1\}} + s_{\{2,2,1,1,1\}} - 2 s_{\{2,2,2,1\}} - 3 s_{\{3,1,1,1,1\}} + s_{\{3,2,1,1\}}$$

$$\text{\texttt{[4, { }3, { }1, { }1]:{ }}} -12 s_{\{1,1,1,1,1,1,1,1\}} + 6 s_{\{2,1,1,1,1,1,1\}} + 4 s_{\{2,2,1,1,1,1,1\}} - s_{\{2,2,2,1,1,1\}} - 4 s_{\{2,2,2,2,1\}} - 6 s_{\{3,1,1,1,1,1,1\}} - s_{\{3,2,1,1,1,1\}} + 3 s_{\{3,2,2,1,1\}} + 2 s_{\{3,2,2,2\}} - s_{\{3,3,1,1,1\}} - s_{\{3,3,2,1\}} + 3 s_{\{4,1,1,1,1,1\}} - s_{\{4,2,1,1,1\}} - s_{\{4,2,2,1\}} + s_{\{4,3,1,1\}}$$

Elementary:

$$\text{\texttt{[3, { }2, { }2]:{ }}} s_{\{1,1,1,1,1,1,1\}} + 2 s_{\{2,1,1,1,1,1\}} + 3 s_{\{2,2,1,1,1\}} + 2 s_{\{2,2,2,1\}} + s_{\{3,1,1,1,1\}} + 2 s_{\{3,2,1,1\}} + s_{\{3,2,2\}} + s_{\{3,3,1\}}$$

$$\text{\texttt{[3, { }2, { }1, { }1]:{ }}} s_{\{1,1,1,1,1,1,1\}} + 3 s_{\{2,1,1,1,1,1\}} + 4 s_{\{2,2,1,1,1\}} + 3 s_{\{2,2,2,1\}} + 3 s_{\{3,1,1,1,1\}} + 4 s_{\{3,2,1,1\}} + 2 s_{\{3,2,2\}} + s_{\{3,3,1\}} + s_{\{4,1,1,1\}} + s_{\{4,2,1\}}$$

$$\text{\texttt{[4, { }3, { }1, { }1]:{ }}} s_{\{1,1,1,1,1,1,1,1\}} + 3 s_{\{2,1,1,1,1,1,1\}} + 4 s_{\{2,2,1,1,1,1,1\}} + 4 s_{\{2,2,2,1,1,1\}} + 3 s_{\{2,2,2,2,1\}} + 3 s_{\{3,1,1,1,1,1,1\}} + 4 s_{\{3,2,1,1,1,1\}} + 4 s_{\{3,2,2,1,1\}} + 2 s_{\{3,2,2,2\}} + s_{\{3,3,1,1,1\}} + s_{\{3,3,2,1\}} + s_{\{4,1,1,1,1,1\}} + s_{\{4,2,1,1,1\}} + s_{\{4,2,2,1\}}$$

Homogeneous:

$$\text{\texttt{[3, { }2, { }2]:{ }}} s_{\{3,2,2\}} + s_{\{3,3,1\}} + 2 s_{\{4,2,1\}} + 2 s_{\{4,3\}} + s_{\{5,1,1\}} + 3 s_{\{5,2\}} + 2 s_{\{6,1\}} + s_{\{7\}}$$

$$\text{\texttt{[3,{ }2,{ }1,{ }1]:{ }}} s_{\{3,2,1,1\}} + s_{\{3,2,2\}} + 2 s_{\{3,3,1\}} + s_{\{4,1,1,1\}} + 4 s_{\{4,2,1\}} + 3 s_{\{4,3\}} + 3 s_{\{5,1,1\}} + 4 s_{\{5,2\}} + 3 s_{\{6,1\}} + s_{\{7\}}$$

$$\text{\texttt{[4,{ }3,{ }1,{ }1]:{ }}} s_{\{4,3,1,1\}} + s_{\{4,3,2\}} + 2 s_{\{4,4,1\}} + s_{\{5,2,1,1\}} + s_{\{5,2,2\}} + 4 s_{\{5,3,1\}} + 3 s_{\{5,4\}} + s_{\{6,1,1,1\}} + 4 s_{\{6,2,1\}} + 4 s_{\{6,3\}} + 3 s_{\{7,1,1\}} + 4 s_{\{7,2\}} + 3 s_{\{8,1\}} + s_{\{9\}}$$

Schur:

$$\text{\texttt{[3,{ }2,{ }2]:{ }}} s_{\{3,2,2\}}$$

$$\text{\texttt{[3,{ }2,{ }1,{ }1]:{ }}} s_{\{3,2,1,1\}}$$

$$\text{\texttt{[4,{ }3,{ }1,{ }1]:{ }}} s_{\{4,3,1,1\}}$$

Powersum:

$$\text{\texttt{[3,{ }2,{ }2]:{ }}} s_{\{1,1,1,1,1,1\}} - s_{\{2,1,1,1,1\}} + 2 s_{\{2,2,1,1\}} - s_{\{2,2,2,1\}} - s_{\{3,1,1,1,1\}} - s_{\{3,2,1,1\}} + s_{\{3,2,2\}} + s_{\{3,3,1\}} + 2 s_{\{4,1,1,1\}} - s_{\{4,2,1\}} - s_{\{4,3\}} - s_{\{5,1,1\}} + 2 s_{\{5,2\}} - s_{\{6,1\}} + s_{\{7\}}$$

$$\text{\texttt{[3,{ }2,{ }1,{ }1]:{ }}} -s_{\{1,1,1,1,1,1\}} - s_{\{2,1,1,1,1\}} - s_{\{2,2,2,1\}} + s_{\{3,1,1,1,1\}} + s_{\{3,2,1,1\}} - s_{\{3,2,2\}} + s_{\{3,3,1\}} - s_{\{4,2,1\}} + s_{\{4,3\}} - s_{\{5,1,1\}} + s_{\{6,1\}} + s_{\{7\}}$$

$$\text{\texttt{[4,{ }3,{ }1,{ }1]:{ }}} -s_{\{1,1,1,1,1,1,1\}} - s_{\{2,1,1,1,1,1,1\}} + s_{\{2,2,1,1,1,1,1\}} - 2 s_{\{2,2,2,2,1\}} + s_{\{3,2,1,1,1,1\}} + s_{\{3,3,2,1\}} - s_{\{4,1,1,1,1,1\}} - s_{\{4,2,1,1,1\}} + s_{\{4,2,2,1\}} - s_{\{4,3,1,1\}} - s_{\{4,3,2\}} + s_{\{5,2,1,1\}} + 2 s_{\{5,4\}} + s_{\{6,1,1,1\}} - s_{\{6,2,1\}} - s_{\{7,2\}} + s_{\{8,1\}} + s_{\{9\}}$$

Looks like the schur, elementary, homogeneous bases appear to be Schur-positive. All three expansions in those bases have positive coefficients.

Exercise 2: Verify the identities of Exercise 1 on p.74 of Fulton for values of n up to at least 5.

$$\text{\texttt{Identity 1: }} n e_n - p_1 e_{n-1} + p_2 e_{n-2} - \dots + (-1)^n p_n = \sum_{r=1}^n (-1)^{r-1} p_r e_{n-r} = 0$$

$$\text{\texttt{Identity 2: }} n h_n - p_1 h_{n-1} - p_2 h_{n-2} - \dots - p_n = \sum_{r=1}^n p_r h_{n-r} = 0$$

```
In [27]: for n in range(1,6):
          #1
          expr1 = n*e[n] + sum(
              (-1)^r * p[r] * e[n-r]
              for r in range(1,n+1))
```

```

)

#2
expr2 = n*h[n] - sum(
    p[r] * h[n-r]
    for r in range(1,n+1)
)

print(f"For n = {n}:")

print(f"Elementary identity is {expr1 == 0}:")

print(f"Homogeneous identity is {expr2 == 0}")

print("-----")

```

```

For n = 1:
Elementary identity is True:
Homogeneous identity is True
-----
For n = 2:
Elementary identity is True:
Homogeneous identity is True
-----
For n = 3:
Elementary identity is True:
Homogeneous identity is True
-----
For n = 4:
Elementary identity is True:
Homogeneous identity is True
-----
For n = 5:
Elementary identity is True:
Homogeneous identity is True
-----

```

Exercise 3: Verify Equation (5) on p.75 of Fulton for $\lambda=[2,1]$ and another larger partition of your choice.

Equation 5: $s_{\lambda}(x) = \det(h_{\lambda_i + j - i}(x))_{1 \leq i, j \leq k}$

```

In [29]: partitions = [[2,1], [3,1,1]]

for lamda in partitions:

    k = len(lamda)

    expr = matrix([
        [
            h[lamda[i] + j - i] if lamda[i] + j - i >= 0 else 0

```

```

        for j in range(k)
    ]
    for i in range(k)
]).det()

print(f"For lambda = {lamda}:")

print(
    f"Equation 5 is {s[lamda] == expr}"
)

print("-----")

```

```

For lambda = [2, 1]:
Equation 5 is True
-----
For lambda = [3, 1, 1]:
Equation 5 is True
-----

```

Exercise 4: Sage has a command for the inner product $\langle f, g \rangle$: `f.scalar(g)`.

- Compute some examples of this product on various symmetric functions. Make sure you find some examples for which the inner product is not 0 or 1.
- Compute an example that demonstrates the symmetry: $\langle f, g \rangle = \langle g, f \rangle$
- Compute examples demonstrating that its action on the Schur basis matches the definition given in class and the book.
- Compute examples demonstrating properties 3. and 4. from the Theorem in class (about the action on the p, m, h bases).

non-1 or 0 examples:

```
In [38]: p([2,1]).scalar([2,1])
```

```
Out[38]: 2
```

```
In [48]: h([3,2,1]).scalar(h([3,2,1]))
```

```
Out[48]: 12
```

inner product symmetry example:

```
In [49]: e([2,2,1]).scalar(h([2,2,1]))
```

Out[49]: 5

```
In [50]: h([2,2,1]).scalar(e([2,2,1]))
```

Out[50]: 5

Demonstrating that $\langle s_{\lambda}, s_{\mu} \rangle = \delta_{\lambda\mu} = \begin{cases} 1 & \text{if } \lambda = \mu, \\ 0 & \text{otherwise} \end{cases}$.

```
In [53]: print("<s[2,1], s[2,1]> =")
print(s[2,1].scalar(s[2,1]))

print("<s[2,1], s[3]> =")
print(s[2,1].scalar(s[3]))
```

```
<s[2,1], s[2,1]> =
1
<s[2,1], s[3]> =
0
```

Demonstrating that $\langle p_{\lambda}, p_{\mu} \rangle = \delta_{\lambda\mu} z_{\lambda}$; (3)

$\langle h_{\lambda}, m_{\mu} \rangle = \delta_{\lambda\mu}$; (4)

```
In [59]: #kronecker delta
def delta(lam, mu):
    return int(lam == mu)

# z_lambda
def z(lamda):
    return Partition(lamda).centralizer_size()

pairs = [
    ([3], [3]),
    ([3], [2,1]),
    ([2,1], [2,1]),
    ([1,1,1], [1,1,1])
]
# (3)
for lam, mu in pairs:

    print(f"{p[lam].scalar(p[mu])}=<p{lam},p{mu}> = delta({lam},{mu})*z({lan
print("-----")

# (4)

for lam, mu in pairs:
```

```

print(f"{h[lam].scalar(m[mu])}<=h{lam},m{mu}> = delta({lam},{mu}) is {h[
3=<p[3],p[3]> = delta([3],[3])*z([3]) is True
0=<p[3],p[2, 1]> = delta([3],[2, 1])*z([3]) is True
2=<p[2, 1],p[2, 1]> = delta([2, 1],[2, 1])*z([2, 1]) is True
6=<p[1, 1, 1],p[1, 1, 1]> = delta([1, 1, 1],[1, 1, 1])*z([1, 1, 1]) is True
-----
1=<h[3],m[3]> = delta([3],[3]) is True
0=<h[3],m[2, 1]> = delta([3],[2, 1]) is True
1=<h[2, 1],m[2, 1]> = delta([2, 1],[2, 1]) is True
1=<h[1, 1, 1],m[1, 1, 1]> = delta([1, 1, 1],[1, 1, 1]) is True

```

Exercise 5: Sage has a command for the ω involution: `.omega()`.

- Compute some examples of the properties discussed in class of the action of the omega involution on the s, h, e bases.
- Demonstrate that ω is an involution.
- Demonstrate that ω is an isometry.

1. $\omega(s_{\lambda}) = s_{\lambda'}$
2. $\omega(h_{\lambda}) = e_{\lambda}$
3. $\omega(e_{\lambda}) = h_{\lambda}$
4. Involution: $\omega^2(f) = f$
5. Isometry: $\langle \omega(f), \omega(g) \rangle = \langle f, g \rangle$

```

In [ ]: from IPython.display import Markdown, display

display(Markdown(r"1.  $\omega(s_{\lambda}) = s_{\lambda'}$ "))

for lamda in [[3], [2,1], [3,1]]:

    print(f"For lambda = {lamda}:")

    print(
        s[lamda].omega()
        ==
        s[Partition(lamda).conjugate()]
    )

    print("-----")

display(Markdown(r"2.  $\omega(h_{\lambda}) = e_{\lambda}$ "))

```

```

for lamda in [[3], [2,1], [1,1,1]]:

    print(f"For lambda = {lamda}:")

    print(
        h[lamda].omega()
        ==
        e[lamda]
    )

    print("-----")

display(Markdown(r"3.  $\omega(e_\lambda)=h_\lambda$ "))

for lamda in [[3], [2,1], [1,1,1]]:

    print(f"For lambda = {lamda}:")

    print(
        e[lamda].omega()
        ==
        h[lamda]
    )

    print("-----")

display(Markdown(r"4.  $\omega^2(f)=f$ "))
print("For f = s[3] + 2*s[2,1] + h[2,1]")

f = s[3] + 2*s[2,1] + h[2,1]

print(f.omega().omega() == f)

print("-----")

display(Markdown(r"5.  $\langle \omega(f), \omega(g) \rangle = \langle f, g \rangle$ "))
print("For f = s[3] + 2*s[2,1] and g = h[3] + e[2,1]")
f = s[3] + 2*s[2,1]
g = h[3] + e[2,1]

print(
    f.omega().scalar(g.omega())
    ==
    f.scalar(g)
)

```

1. $\omega(s_\lambda)=s_{\lambda'}$


```
For lambda = [3]:
```

```
True
```

```
-----
```

```
For lambda = [2, 1]:
```

```
True
```

```
-----
```

```
For lambda = [3, 1]:
```

```
True
```

```
-----
```

2. $\omega(h_\lambda)=e_\lambda$

```
For lambda = [3]:
```

```
True
```

```
-----
```

```
For lambda = [2, 1]:
```

```
True
```

```
-----
```

```
For lambda = [1, 1, 1]:
```

```
True
```

```
-----
```

3. $\omega(e_\lambda)=h_\lambda$

```
For lambda = [3]:
```

```
True
```

```
-----
```

```
For lambda = [2, 1]:
```

```
True
```

```
-----
```

```
For lambda = [1, 1, 1]:
```

```
True
```

```
-----
```

4. $\omega^2(f)=f$

```
For f = s[3] + 2*s[2,1] + h[2,1]
```

```
True
```

```
-----
```

5. $\langle \omega(f), \omega(g) \rangle = \langle f, g \rangle$

```
For f = s[3] + 2*s[2,1] and g = h[3] + e[2,1]
```

```
True
```